I'll explain your code line by line for both files:

Calculator.java Explanation

```
java

package labExam1;
```

- Declares the package - organizes related classes together

```
java

import java.util.ArrayList;

import java.util.Scanner;
```

- Imports necessary classes:
  - `ArrayList` - for dynamic array functionality
  - `Scanner` - for user input handling

```
java

public class Calculator {
```

- Class declaration - defines a public class named "Calculator"

```
java

   private ArrayList<String> history = new ArrayList<>();
```

- Instance variable - creates a private ArrayList to store calculation history as strings
- `private` - only accessible within this class
- `ArrayList<String>` - generic type that stores only String objects
- `new ArrayList<>()` - instantiates a new empty ArrayList

```
java

   private Scanner input = new Scanner(System.in);
```

- Instance variable - creates a Scanner object to read input from system console

```
java
```

```
// 1. Function to print odd integers from range a to b
```

```
public void productCalculate(int a, int b){
```

- Method declaration - public method that takes two integers and returns nothing (void)
- Note: The method name `productCalculate` is misleading since it doesn't calculate products

```
java
```

```
System.out.println("Odd integers from " + a + " to " + b + ":");
```

- Output - prints the range being processed with string concatenation

```
java
```

```
for (int i = a; i <= b; i++){
```

- Loop - iterates from `a` to `b` (inclusive)

```
java
```

```
if(i % 2 != 0){
```

- Condition - checks if number is odd using modulus operator
- `i % 2` gives remainder when divided by 2
- `!= 0` means not equal to zero (odd number)

```
java
```

```
System.out.print(i + " ");
```

- Output - prints the odd number followed by space (on same line)

```
java
```

```
System.out.println();
```

- Output - prints a newline after all odd numbers are displayed

```
java
```

```
history.add("Printed odd numbers from " + a + " to " + b);
```

- History tracking - adds this operation to the history ArrayList

```
java
```

```
   // 2. Method overloading for sumCalculate with 2 parameters
```

```
    public int sumCalculate(int num1, int num2) {
```

- Overloaded method - same name but different parameters than other sumCalculate methods

```
java
```

```
        int result = num1 + num2;
```

- Calculation - stores sum of two numbers in local variable

```
java
```

```
        history.add(num1 + " + " + num2 + " = " + result);
```

- History tracking - adds the calculation with result to history

```
java
```

```
        return result;
```

- Return value - sends the calculated sum back to caller

```
java
```

```
   // 2. Method overloading for sumCalculate with 3 parameters
```

```
    public int sumCalculate(int num1, int num2, int num3) {
```

- Method overloading - same method name but with 3 parameters

```
java
```

```
   // 2. Method overloading for sumCalculate with 4 parameters
```

```
    public int sumCalculate(int num1, int num2, int num3, int num4) {
```

- Method overloading - same method name but with 4 parameters

```
java
```

```
   // 3. Function to display calculation history
```

```
    public void calculationHistory() {
```

- Method declaration - displays calculation history based on user choice

```
java
        System.out.print("Do you want to see calculation history? (yes/no): ");
```

- User prompt - asks user if they want to see history

```
java
        String choice = input.nextLine().toLowerCase();
```

- Input handling:
  - `input.nextLine()` - reads entire line of user input
  - `toLowerCase()` - converts input to lowercase for case-insensitive comparison

```
java
        if (choice.equals("yes") || choice.equals("y")) {
```

- Condition - checks if user entered "yes" or "y"

```
java
            if (history.isEmpty()) {
```

- Condition - checks if history ArrayList is empty

```
java
                System.out.println("No calculations in history yet.");
```

- Output - informs user that no calculations exist

```
java
            } else {
                System.out.println("\n=== Calculation History ===");
```

- Output - prints header for history display

```
java
                for (int i = 0; i < history.size(); i++) {
```

- Loop - iterates through all elements in history ArrayList

```
java
                    System.out.println((i + 1) + ". " + history.get(i));
```

- Output - displays each history item with numbering
- `i + 1` - converts zero-based index to one-based numbering
- `history.get(i)` - retrieves element at position i

```
java
                System.out.println("============================\n");
```

- Output - prints footer after history display

Main.java Explanation

```
java
package labExam1;
```

- Package declaration - same package as Calculator class

```
java
import java.util.Scanner;
```

- Import - Scanner class for user input

```
java
public class Main {
```

- Main class - entry point of the application

```
java
```

```
    // Main method to test the calculator
    public static void main(String[] args) {
```

- Main method - entry point that JVM calls to start program
- `public` - accessible from anywhere
- `static` - belongs to class, not instance
- `void` - doesn't return anything
- `String[] args` - command line arguments

```java
        Calculator calculator = new Calculator();
```

- Object creation - creates an instance of Calculator class

```java
        Scanner mainScanner = new Scanner(System.in);
```

- Scanner creation - for reading user input (though not used in this code)

```java
        System.out.println("=== Java Calculator ===");
```

- Output - displays program header

```java
        // Test productCalculate
        System.out.println("\nTesting productCalculate:");
        calculator.productCalculate(1, 10);
```

- Method call - tests the odd number printing functionality with range 1-10

```java
        // Test sumCalculate with different numbers of parameters
        System.out.println("\nTesting sumCalculate with 2 numbers:");
        int result2 = calculator.sumCalculate(5, 10);
        System.out.println("5 + 10 = " + result2);
```

- Method overloading demo - calls 2-parameter version and displays result

```java
        System.out.println("\nTesting sumCalculate with 3 numbers:");
         int result3 = calculator.sumCalculate(5, 10, 15);

         System.out.println("5 + 10 + 15 = " + result3);
```

- Method overloading demo - calls 3-parameter version

```java
        System.out.println("\nTesting sumCalculate with 4 numbers:");
         int result4 = calculator.sumCalculate(5, 10, 15, 20);

         System.out.println("5 + 10 + 15 + 20 = " + result4);
```

- Method overloading demo - calls 4-parameter version

```java
        // Test calculationHistory

         calculator.calculationHistory();
```

- Method call - displays history based on user input

```java
        mainScanner.close();
```

- Resource cleanup - closes the Scanner to release system resources

Key Concepts Demonstrated:

1. Encapsulation - private fields with public methods
2. Method Overloading - multiple `sumCalculate` methods with different parameters
3. Collections - using ArrayList to store history
4. User Input - Scanner for interactive functionality
5. Control Structures - loops and conditionals
6. String Manipulation - concatenation and formatting

The code successfully implements all required features from the lab exam specification.